

Звіт

до Лабораторної роботи №6

Поліморфізм з абстрактними класами в Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок створення поліморфних моделей у Java з використанням абстрактних класів.

У ході роботи необхідно створити абстрактний базовий клас, виділити спільний стан і спільну поведінку, оголосити абстрактний метод, реалізувати конкретні підкласи та продемонструвати поліморфний виклик через базовий тип.

2. Опис предметної області

У даній лабораторній роботі розглядається предметна область способів доставки. У системі можуть існувати різні способи доставки замовлення: кур'єрська доставка, доставка до поштомата та доставка дроном.

Усі способи доставки мають спільні дані: номер замовлення, адресу доставки та базову вартість. Але кінцева вартість доставки для кожного типу розраховується по-різному.

Тому для цієї задачі доцільно використати абстрактний клас `DeliveryMethod`, а конкретні способи доставки реалізувати як його підкласи.

Побудована ієрархія:

DeliveryMethod

├── CourierDelivery

├── ParcelLockerDelivery

└── DroneDelivery

3. Побудова ієрархії класів

3.1 Абстрактний клас DeliveryMethod

Клас DeliveryMethod є абстрактним базовим класом. Він описує загальне поняття способу доставки.

Сам по собі об'єкт DeliveryMethod створювати недоцільно, тому що це занадто загальне поняття. У реальній програмі завжди використовується конкретний спосіб доставки.

Спільні поля класу:

- orderId — номер замовлення;
- address — адреса доставки;
- basePrice — базова вартість доставки.

Також у базовому класі реалізовано метод getInfo(), який повертає коротку інформацію про доставку.

Основний абстрактний метод:

```
public abstract double calculatePrice();
```

3.2 Клас CourierDelivery

Клас CourierDelivery описує кур'єрську доставку. Він є конкретним підкласом абстрактного класу DeliveryMethod.

Власне поле класу:

- distanceKm — відстань доставки в кілометрах.

Метод calculatePrice() розраховує вартість доставки з урахуванням відстані:

$$\text{basePrice} + \text{distanceKm} * 10$$

3.3 Клас ParcelLockerDelivery

Клас ParcelLockerDelivery описує доставку до поштомата. Він також є конкретним підкласом DeliveryMethod.

Власне поле класу:

- lockerNumber — номер поштомата.

Метод calculatePrice() додає фіксовану доплату до базової ціни:

$\text{basePrice} + 20$

3.4 Клас DroneDelivery

Клас DroneDelivery описує доставку дроном. Він є третім конкретним підкласом абстрактного типу DeliveryMethod.

Власне поле класу:

- weightKg — вага посилки.

Метод calculatePrice() розраховує вартість доставки з урахуванням ваги:

$\text{basePrice} + \text{weightKg} * 30$

4. Обґрунтування абстракції

Абстрактний клас DeliveryMethod є семантично коректним, тому що він описує спільне поняття для різних способів доставки.

Кур'єрська доставка, доставка до поштомата та доставка дроном мають однаковий загальний зміст — вони доставляють замовлення. Але кожен спосіб має власні особливості та власний спосіб розрахунку вартості.

Через це спільні поля та частину спільної поведінки винесено до базового абстрактного класу, а різну поведінку реалізовано в підкласах через перевизначення методу calculatePrice().

5. Спільний стан і спільна поведінка

До абстрактного класу DeliveryMethod було перенесено спільний стан, який потрібен усім способам доставки.

Елемент	Опис
orderId	номер замовлення
address	адреса доставки
basePrice	базова вартість доставки
getInfo()	метод для отримання короткої інформації про доставку
calculatePrice()	абстрактний метод, який визначає поліморфний контракт

6. Спеціалізація підкласів

Кожен конкретний підклас має власний стан і власну реалізацію методу calculatePrice().

Клас	Власний стан	Реалізація calculatePrice()
CourierDelivery	distanceKm	basePrice + distanceKm * 10
ParcelLockerDelivery	lockerNumber	basePrice + 20
DroneDelivery	weightKg	basePrice + weightKg * 30

У кожному підкласі використано анотацію `@Override`, яка показує, що метод базового класу перевизначається.

7. Поліморфізм у клієнтському коді

Поліморфізм продемонстровано у класі `Main`. У ньому об'єкти різних конкретних класів зберігаються в масиві базового типу `DeliveryMethod`.

```
DeliveryMethod[] deliveries = {  
    new CourierDelivery("ORD-001", "Харків, вул. Сумська, 20", 100, 5),  
    new ParcelLockerDelivery("ORD-002", "Харків, проспект Науки, 10", 80,  
        "A-15"),  
    new DroneDelivery("ORD-003", "Харків, вул. Кирпичова, 2", 150, 2.5)  
};
```

Далі в циклі викликаються методи через змінну базового типу:

```
for (DeliveryMethod delivery : deliveries) {  
    System.out.println(delivery.getInfo());  
    System.out.println("Price: " + delivery.calculatePrice());  
}
```

Це є реальним поліморфним викликом. Змінна `delivery` має тип `DeliveryMethod`, але Java під час виконання викликає реалізацію методу `calculatePrice()` саме того класу, об'єкт якого реально знаходиться в масиві.

8. UML-діаграма класів

Для створеної моделі було побудовано UML-діаграму. На ній показано абстрактний клас `DeliveryMethod`, конкретні підкласи та зв'язки успадкування.

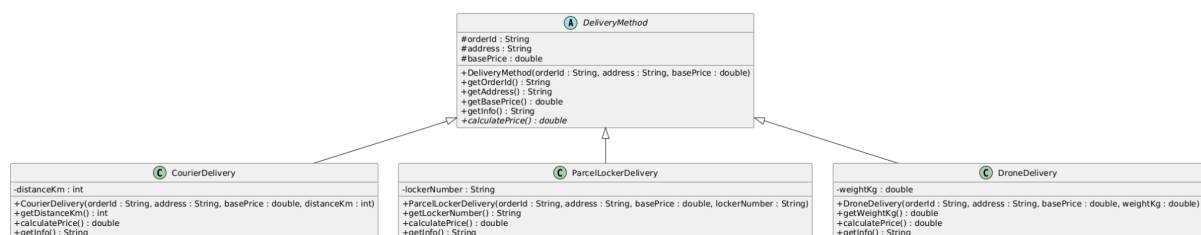


Рисунок 1 — UML-діаграма поліморфної моделі способів доставки

9. Демонстрація роботи програми

Після запуску класу Main програма створює три різні способи доставки та виводить інформацію про кожен із них.

Приклад результату роботи:

Order: ORD-001, address: Харків, вул. Сумська, 20, base price: 100.0, courier distance: 5 km

Price: 150.0

Order: ORD-002, address: Харків, проспект Науки, 10, base price: 80.0, locker number: A-15

Price: 100.0

Order: ORD-003, address: Харків, вул. Кирпичова, 2, base price: 150.0, drone package weight: 2.5 kg

Price: 225.0

```
Order: ORD-001, address: Харків, вул. Сумська, 20, base price: 100.0, courier distance: 5 km
Price: 150.0

Order: ORD-002, address: Харків, проспект Науки, 10, base price: 80.0, locker number: A-15
Price: 100.0

Order: ORD-003, address: Харків, вул. Кирпичова, 2, base price: 150.0, drone package weight: 2.5 kg
Price: 225.0
```

Рисунок 2 — результат запуску програми

10. Тестування

Для перевірки роботи програми були створені тести TestNG.

Тестові класи:

- CourierDeliveryTest;
- ParcelLockerDeliveryTest;
- DroneDeliveryTest;
- DeliveryMethodPolymorphismTest.

Тести перевіряють збереження спільних полів, власні поля підкласів, правильність розрахунку вартості, роботу методу `getInfo()` та поліморфний виклик через базовий тип `DeliveryMethod`.

```
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.431 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
```

Рисунок 3 — результат виконання тестів

11. Додаткове завдання: аналіз моделі з ЛР2

У лабораторній роботі №2 використовувалась модель спортивного клубу. У ній були класи `SportsClub`, `Student`, `Coach`, `SportSection`, `Schedule` та `Enrollment`.

У цій моделі можна спробувати ввести поліморфну абстракцію для сутностей, які описують людей. Найбільш логічними кандидатами є класи `Student` і `Coach`, тому що обидва вони описують людину в системі спортивного клубу.

Можлива ієрархія:

Person

└── Student

└── Coach

Абстрактний клас `Person` міг би містити спільний стан:

- `fullName` — повне ім'я людини.

Також у ньому можна було б оголосити абстрактний метод:

```
public abstract String getRoleInfo();
```

Клас `Student` реалізував би цей метод як інформацію про студента, а клас `Coach` — як інформацію про тренера.

Класи `SportsClub`, `SportSection`, `Schedule` і `Enrollment` не слід включати до цієї ієрархії, тому що вони не є різновидами людини. Вони мають інші ролі в системі та краще моделюються через асоціацію, агрегацію або композицію.

12. Висновок

У ході виконання лабораторної роботи було створено поліморфну модель для варіанта 5 — способи доставки.

У програмі реалізовано абстрактний клас `DeliveryMethod`, який містить спільний стан і спільну поведінку для всіх способів доставки.

Було створено три конкретні підкласи: `CourierDelivery`, `ParcelLockerDelivery` і `DroneDelivery`.

Кожен підклас перевизначає метод `calculatePrice()` і реалізує власний спосіб розрахунку вартості доставки.

У класі `Main` було продемонстровано реальний поліморфний виклик через базовий тип `DeliveryMethod`.

Також було створено UML-діаграму та тести для перевірки роботи класів. Додатково було проаналізовано модель з лабораторної роботи №2 і зроблено висновок, що в ній можна коректно ввести абстрактний базовий клас `Person` для класів `Student` і `Coach`.

Отже, лабораторна робота показала різницю між простим успадкуванням і поліморфізмом: у поліморфній моделі клієнтський код працює через один базовий тип, а конкретна поведінка визначається фактичним типом об'єкта під час виконання програми.